



TRABAJO PRÁCTICO GRUPAL (TPG)

Software embarcado de una nave interestelar tripulada



ANALIZAR

Entender el dominio y sus requerimientos



DISEÑAR

Modelar y documentar la solución



CONSTRUIR

Implementar con calidad y buenas prácticas



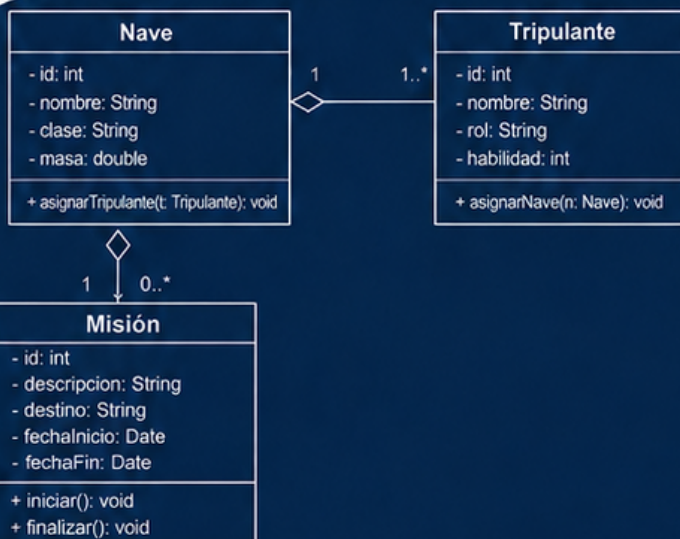
EVOLUCIONAR

Incorporar cambios y nuevas funcionalidades



DEFENDER

Justificar decisiones y comunicar soluciones



GRUPO: _____



FECHA: _____



DOCENTE: _____

MANUAL UNIFICADO Y COMPENDIO DE REFERENCIA

Trabajo Práctico Grupal 2026 — Programación C
Software Embarcado de una Nave Interestelar Tripulada

Organización del Compendio: Orden Secuencial de Lectura Oficial (LEAME PRIMERO)

Fecha de Generación: Septiembre 2026

Destinatarios: Cátedra de Programación C y Alumnos del TPG

Sumario

I. Guía General para Estudiantes (v0.4)	6
1. Presentación	6
2. El problema que debe resolver el equipo	6
3. Participantes y responsabilidades	6
3.1 Tripulación y capitán/a	6
3.2 Asistente de navegación y comando	7
3.3 Nave y subsistemas	7
3.4 Motor de Simulación y biblioteca provista	7
4. Evolución del trabajo	7
4.1 Entrega 1: construcción del núcleo orientado a objetos	7
4.2 Entrega 2: operación dentro de un universo dinámico	8
5. Universo dinámico	8
6. Navegación y toma de decisiones	9
7. Misiones obligatorias	9
7.1 M-01 – Intercepción y asistencia	9
7.2 M-02 – Recolección en un punto de interés	9
7.3 M-03 – Retorno seguro a una zona designada	9
8. Recursos, mantenimiento e informes	10
9. Qué queda fuera del alcance obligatorio	10
10. Organización del proyecto y forma de trabajo	11
11. Cómo leer la documentación	11
12. Síntesis del desafío	11
II. Enunciado de la Entrega 1	12
1. Propósito de la entrega	12
2. Alcance	12
3. Requerimientos	12
E1-01 – Modelo de Nave	12
E1-02 – Motor Warp mediante State	13
E1-03 – Asistente de Comando	13
E1-04 – Tripulación	13
E1-05 – Bitácora	13
E1-06 – Tres misiones simplificadas mediante Template Method	14
E1-07 – Creación de naves mediante Factory	14

E1-08 – Liquidación de haberes mediante Decorator.....	14
E1-09 – Recursos y mantenimiento.....	15
E1-10 – Informe mínimo de misión.....	15
4. Patrones obligatorios.....	15
5. Contratos mínimos.....	15
6. Evidencias funcionales mínimas.....	15
7. Herramientas y entrega.....	16
8. Criterios mínimos de aprobación.....	16
III. Ficha de Inicio – Entrega 1.....	17
1. Propósito.....	17
2. Configuración inicial de las naves.....	17
3. Tripulación mínima.....	17
4. Misiones obligatorias.....	17
5. Costos de la misión simplificada.....	18
6. Escenarios mínimos que debe demostrar el grupo.....	18
Escenario A – Ejecución correcta.....	18
Escenario B – Recursos insuficientes.....	18
Escenario C – Motor Warp.....	18
Escenario D – Contrato inválido.....	19
7. Lista de comprobación antes de entregar.....	19
IV. Tabla de Recursos, Costos y Consecuencias.....	20
1. Criterio general.....	20
2. Costos comunes.....	20
3. Consecuencias del universo.....	20
4. Acción final de cada misión.....	21
5. Cálculo total.....	21
6. Ejemplo breve.....	21
7. Regla de simplicidad.....	21
V. Reglas Operativas del TPG 2026.....	22
1. Integración de los grupos.....	22
2. Responsabilidad del equipo.....	22
3. Repositorio GitHub.....	22
4. Identificación de las entregas.....	22
5. Contenido mínimo del repositorio.....	23
6. Participación y autoría.....	23
7. Uso permitido de inteligencia artificial.....	23
8. Registro del uso de IA.....	24

9. Usos no permitidos.....	24
10. Defensa.....	24
11. Resultado de la defensa.....	25
12. Integridad y fuentes externas.....	25
13. Regla de prevalencia.....	25
VI. Rúbrica – Entrega 1.....	26
1. Propósito.....	26
2. Criterios y niveles de desempeño.....	26
3. Condiciones mínimas de aprobación.....	27
4. Límites de calificación por incumplimientos centrales.....	28
5. Evidencia mínima para la corrección.....	28
6. Regla de corrección.....	28

I. Guía General para Estudiantes (v0.4)

Asignatura: Programación C

Dominio: software embarcado de una nave interestelar tripulada

Versión de la guía: v0.4

Alcance técnico de referencia: TPG v0.3

1. Presentación

En un futuro en el que la exploración espacial se ha extendido a regiones cada vez más alejadas, las naves interestelares deben desarrollar misiones durante períodos prolongados y en entornos que pueden cambiar de manera imprevisible.

Las grandes distancias hacen que las comunicaciones con las bases de operaciones no siempre sean inmediatas. Por esta razón, cada nave cuenta con una tripulación comandada por un/a capitán/a y con un software embarcado capaz de asistirle durante la preparación, ejecución y evaluación de sus misiones.

El sistema que desarrollará cada equipo funcionará como asistente de navegación y comando. Su responsabilidad será ayudar a la tripulación a administrar la nave, coordinar algunos de sus subsistemas, controlar el consumo de recursos, registrar los acontecimientos importantes y ejecutar las acciones necesarias para cumplir una misión.

El asistente no reemplaza al/a la capitán/a ni controla libremente la nave. Opera dentro de reglas y autorizaciones previamente establecidas. Algunas acciones podrán ejecutarse automáticamente cuando exista una regla explícita que así lo determine; otras deberán presentarse como recomendaciones o quedar registradas para conocimiento de la tripulación.

La nave dispone de recursos limitados, entre ellos combustible y energía. Su utilización produce desgaste y puede generar necesidades de mantenimiento. Por ello, antes de iniciar o continuar una operación, el sistema deberá comprobar que la nave se encuentre en condiciones adecuadas y que disponga de los recursos necesarios.

2. El problema que debe resolver el equipo

El objetivo es construir y hacer evolucionar el software embarcado de una nave interestelar tripulada. El sistema deberá ser capaz de representar la nave y su tripulación, administrar subsistemas y recursos, ejecutar misiones, registrar lo ocurrido y asistir en la navegación dentro de un universo dinámico.

El desafío no consiste solamente en lograr que una simulación funcione. La solución deberá estar organizada como un sistema orientado a objetos comprensible, extensible y verificable. Cada equipo deberá poder explicar las responsabilidades de sus clases, justificar los patrones empleados, demostrar el cumplimiento de los contratos y defender las decisiones de diseño adoptadas.

3. Participantes y responsabilidades

3.1 Tripulación y capitán/a

La nave es tripulada. El/la capitán/a conserva la autoridad general sobre la operación y la tripulación cumple las funciones correspondientes a sus cargos. En la primera entrega también se modelarán los datos necesarios de la tripulación y su asociación con la nave.

3.2 Asistente de navegación y comando

Es el software que desarrollará el equipo. Debe:

- recibir y coordinar órdenes permitidas;
- consultar el estado de la nave y sus recursos;
- colaborar en la preparación y ejecución de misiones;
- coordinar subsistemas sin apropiarse de sus responsabilidades internas;
- observar el entorno cuando se incorporen los sensores;
- interpretar eventos y aplicar las reglas de navegación establecidas;
- registrar eventos, órdenes, decisiones y resultados;
- producir informes que permitan reconstruir lo ocurrido.

3.3 Nave y subsistemas

La nave reúne su identidad, tipo, componentes, recursos y estado operativo. Algunos comportamientos serán delegados en subsistemas especializados, como el Motor Warp, la bitácora, los sensores y los mecanismos de persistencia. La nave no debe convertirse en una clase que concentre toda la lógica del sistema.

3.4 Motor de Simulación y biblioteca provista

En la segunda entrega, una biblioteca proporcionada por la cátedra administrará la evolución del universo. Esta biblioteca actuará como Motor de Simulación y ofrecerá el tablero, los snapshots y los eventos necesarios para observar el entorno.

El Motor de Simulación:

- modifica el universo;
- produce información observable;
- no controla la nave;
- no modifica los subsistemas internos de la nave;
- no decide qué acción debe ejecutar el asistente;
- no debe ser implementado ni modificado por los estudiantes.

4. Evolución del trabajo

El TPG se desarrolla mediante dos entregas incrementales. La segunda entrega continúa y amplía el mismo proyecto: no se comienza nuevamente ni se reemplaza la solución anterior.

4.1 Entrega 1: construcción del núcleo orientado a objetos

Durante la primera etapa se trabajará con un entorno controlado, sin integración con el universo dinámico. El equipo deberá construir el núcleo del software embarcado:

- nave y tipos de nave;
- tripulación;
- asistente de comando;
- Motor Warp;
- bitácora;
- recursos y mantenimiento;
- ciclo básico de una misiones;

- informe de misión;
- contratos, invariantes y manejo de errores.

Esta etapa incorpora los patrones State, Factory y Template Method en los casos especificados en el enunciado de la Entrega 1. Aunque la demostración inicial puede realizarse desde un main, el dominio no debe leer ni escribir directamente en consola y no debe conocer Swing. Esta separación permitirá conectar en E2 los mismos objetos con vistas y controladores sin reescribir la lógica.

Cada grupo implementará versiones simplificadas de M-01, M-02 y M-03, desacopladas del universo real. Su finalidad es demostrar un ciclo común de preparación, ejecución, evaluación y cierre, junto con las variaciones propias de cada objetivo. En E2 se ampliarán esas mismas clases con tablero, sensores y eventos.

4.2 Entrega 2: operación dentro de un universo dinámico

En la segunda etapa, el núcleo construido se integrará con la biblioteca provista por la cátedra. El sistema deberá observar un tablero de 20 × 20 mediante snapshots y sensores, interpretar eventos y aplicar reglas explícitas de navegación.

La información seguirá el siguiente recorrido conceptual:

Motor de Simulación → Tablero → Sensores → Sistema de navegación → Nave y subsistemas

Esta separación establece una frontera de responsabilidades:

- el motor hace evolucionar el universo;
- el tablero ofrece una representación observable;
- los sensores leen y detectan;
- el sistema de navegación interpreta y decide;
- la nave y sus subsistemas ejecutan acciones válidas.

Los sensores no deben controlar la nave. Del mismo modo, la nave no debe acceder directamente al Motor de Simulación.

En esta etapa se incorporarán Observer, Double Dispatch, DAO/DTO, JDBC, concurrencia y una interfaz Swing organizada bajo MVC, conforme al enunciado de la Entrega 2. El acceso mediante usuario y contraseña servirá como caso completo para diferenciar validaciones de formato en la Vista, coordinación en el Controlador, reglas y excepciones en los servicios, y consultas en los DAO.

5. Universo dinámico

El universo puede cambiar mientras la nave está operando. El Motor de Simulación podrá comunicar, entre otros, los siguientes acontecimientos:

- aparición de un asteroide;
- presencia de una anomalía gravitacional;
- tormenta electromagnética;
- detección de una nave externa;
- recepción de una señal de auxilio.

El software embarcado deberá transformar la información técnica recibida en eventos significativos para el dominio, evitar duplicaciones y confirmar las condiciones observadas antes de actuar. Un evento no equivale automáticamente a una acción. Primero debe ser interpretado y luego evaluado por las reglas correspondientes.

6. Navegación y toma de decisiones

La navegación se resolverá mediante reglas explícitas. Cada regla relacionará una condición observable con una respuesta permitida. Las cinco situaciones mínimas son:

1. Si se detecta un asteroide mientras la nave se encuentra en warp, se debe abortar el salto.
2. Si una anomalía gravitacional se encuentra próxima a la ruta, se debe recalcular el recorrido.
3. Ante una tormenta electromagnética, se debe reducir la velocidad.
4. Si una nave desconocida se encuentra a corta distancia, se debe activar el protocolo defensivo definido para el TPG.
5. Si se recibe una señal de auxilio y la misión activa no es crítica, se debe registrar el evento y sugerir un desvío.

Estas reglas delimitan el comportamiento obligatorio. No se solicita desarrollar una inteligencia artificial que decida por la nave ni un sistema completo de combate, daños o negociación.

7. Misiones obligatorias

En la segunda entrega se integrarán tres misiones. Todas deberán evaluar su factibilidad, utilizar recursos, reaccionar ante eventos pertinentes, dejar registros y producir un informe final.

7.1 M-01 – Intercepción y asistencia

La nave recibe o detecta una situación que requiere aproximarse a otra nave u objetivo para verificar su estado o brindar asistencia.

Antes de iniciar la misión, el sistema deberá comprobar que el objetivo sea alcanzable, que la nave se encuentre operativa y que existan recursos suficientes. Durante la aproximación podrán aparecer obstáculos, interferencias o cambios en la posición del objetivo.

La misión finalizará cuando se haya establecido contacto y se haya realizado o evaluado la asistencia, o cuando una condición justificada obligue a cancelarla. El informe deberá indicar el estado del objetivo, la ruta adoptada, los recursos consumidos y el resultado obtenido.

7.2 M-02 – Recolección en un punto de interés

La nave debe dirigirse hacia un punto de interés para registrar información o recolectar una muestra definida por el escenario.

El sistema deberá verificar que el punto sea accesible, planificar la aproximación y reservar los recursos necesarios para el desplazamiento, los sensores y el procedimiento de recolección. La presencia de obstáculos, interferencias o cambios en el entorno podrá modificar la ejecución.

La misión finalizará cuando la información o muestra haya sido obtenida y registrada, o cuando la operación resulte inviable. El informe deberá identificar el punto visitado, el resultado de la recolección, la ruta y los recursos utilizados.

7.3 M-03 – Retorno seguro a una zona designada

La nave debe regresar a una zona de referencia priorizando su seguridad y la conservación de recursos.

El sistema deberá evaluar el estado operativo, calcular una ruta posible y comprobar que el combustible y la energía disponibles permitan completar el regreso. Si las condiciones cambian, deberá ajustar la navegación respetando las reglas del dominio.

La misión finalizará cuando la nave alcance la zona designada y cierre correctamente su operación. El informe deberá indicar la ruta utilizada, las contingencias encontradas, los recursos consumidos y el estado final de la nave.

8. Recursos, mantenimiento e informes

Como mínimo, la nave administrará:

- combustible;
- energía;
- desgaste;
- necesidad o estado de mantenimiento.

Los recursos nunca podrán adoptar valores inválidos. Toda operación que los consuma deberá verificar previamente su disponibilidad. El desgaste y el mantenimiento deberán producir consecuencias coherentes sobre la capacidad operativa de la nave.

Al finalizar una misión se generará un informe que permita conocer, como mínimo:

- la misión ejecutada;
- el objetivo evaluado;
- las acciones realizadas;
- la ruta o recorrido adoptado;
- los eventos relevantes;
- los recursos consumidos;
- el resultado final;
- el estado en que quedó la nave;
- las observaciones registradas.

9. Qué queda fuera del alcance obligatorio

Para mantener el trabajo realizable dentro del calendario, no se solicita:

- implementar o modificar el Motor de Simulación;
- desarrollar una IA que controle la nave;
- implementar un sistema completo de combate, daños o negociación;
- persistir el universo, el tablero o el Motor de Simulación;
- reanudar automáticamente misiones interrumpidas;
- incorporar nuevas misiones obligatorias;
- implementar armas, androides u otros subsistemas no incluidos expresamente en los enunciados vigentes;
- utilizar un ORM (mapeo relacional-objeto);
- crear un hilo por cada evento.

Las ampliaciones opcionales sólo podrán realizarse después de completar los requisitos obligatorios y no reemplazarán ninguna evidencia requerida.

10. Organización del proyecto y forma de trabajo

El desarrollo se realizará en Java mediante Maven y se mantendrá en un repositorio GitHub del grupo. Cada integrante deberá participar en la construcción y comprender la solución completa.

Las decisiones relevantes deberán quedar documentadas. Cuando se utilicen herramientas de inteligencia artificial, su intervención se declarará en docs/uso-ia.md, de acuerdo con las reglas operativas del TPG. Durante la defensa no se permitirá utilizar IA.

La ejecución correcta del programa es necesaria, pero no suficiente. También se evaluarán la calidad del modelo, la separación de responsabilidades, la aplicación funcional de los patrones, el cumplimiento de los contratos, las evidencias y la capacidad individual de explicar el trabajo.

11. Cómo leer la documentación

Esta guía presenta el contexto general y explica cómo evoluciona el sistema. No reemplaza los enunciados técnicos ni agrega requisitos.

Orden de lectura recomendado:

1. esta Guía general;
2. 01_Guia_Navegacion_Misiones_y_Consecuencias_v0.1.md; (si estuviere disponible)
3. 02_Tabla_Recursos_Costos_y_Consecuencias_v0.1.md;
4. Entrega_1_Enunciado_TPG_Programacion_C_2026_v0.3.md;
5. Reglas_Operativas_TPG_Programacion_C_2026_v0.3.md;
6. al comenzar la segunda etapa, Entrega_2_Enunciado_TPG_Programacion_C_2026_v0.3.md;
7. documentación de la biblioteca provista por la cátedra.

Ante una diferencia entre esta explicación narrativa y un requisito técnico, prevalece el enunciado vigente de la entrega.

12. Síntesis del desafío

El trabajo comienza construyendo el núcleo de una nave y termina integrándolo en un universo que cambia concurrentemente. La misma solución deberá evolucionar desde un modelo orientado a objetos inicial hacia un sistema con eventos, navegación por reglas, persistencia, concurrencia e interfaz gráfica.

Cada decisión deberá poder responder tres preguntas:

1. ¿Qué objeto es responsable de este comportamiento?
2. ¿Qué contrato debe cumplir?
3. ¿Cómo se demuestra que funciona correctamente?

El resultado esperado es un sistema que no sólo funcione, sino que pueda ser comprendido, verificado, evolucionado y defendido por todos los integrantes del grupo.

II. Enunciado de la Entrega 1

1. Propósito de la entrega

La Entrega 1 tiene como objetivo construir el núcleo orientado a objetos del software embarcado de una nave interestelar tripulada.

El desarrollo deberá establecer una base arquitectónica clara y extensible para incorporar, en la Entrega 2, el universo dinámico, el tablero, los sensores, la concurrencia, la persistencia y la interfaz gráfica.

Fecha de entrega: 15/10/2026.

2. Alcance

La Entrega 1 incluirá:

Nave y tipos de nave.
 Tripulación.
 Liquidación mensual de haberes de la tripulación.
 Asistente de Comando.
 Motor Warp.
 Bitácora.
 Versiones simplificadas de M-01, M-02 y M-03, desacopladas del universo dinámico.
 Combustible, energía, desgaste y mantenimiento.
 Acciones básicas de carga de combustible, carga de energía y mantenimiento.
 Informe mínimo de misión.
 Manejo de errores y excepciones.
 Contratos de clase y método.
 Evidencias funcionales verificables.

La Entrega 1 no incluirá:

biblioteca del universo;
 tablero;
 sensores reales;
 eventos externos dinámicos;
 interacción con otras naves;
 concurrencia;
 persistencia mediante JDBC;
 interfaz gráfica MVC.

3. Requerimientos

E1-01 – Modelo de Nave

El sistema deberá modelar una nave con identidad, tipo, componentes y recursos. Su estado interno deberá mantenerse encapsulado y respetar las invariantes definidas.

Los tipos iniciales serán:

- nave exploradora;
- nave carguero;
- nave de combate.

En E1, las diferencias entre estos tipos podrán limitarse a su configuración inicial de recursos o componentes.

E1-02 – Motor Warp mediante State

El Motor Warp deberá implementar, como mínimo, los siguientes estados:

- Disponible;
- Preparando salto;
- En warp;
- Enfriamiento.

Cada estado determinará qué operaciones y transiciones son válidas. Las transiciones inválidas deberán rechazarse o registrarse explícitamente.

E1-03 – Asistente de Comando

El Asistente de Comando deberá recibir órdenes, coordinarlas con el componente correspondiente, manejar los errores producidos y registrar los resultados en la Bitácora.

El asistente no deberá implementar el comportamiento interno de los subsistemas.

Las operaciones del asistente y del dominio deberán poder ser invocadas posteriormente desde controladores gráficos. Por ello, las clases del núcleo no dependerán de Scanner, System.out, JOptionPane, JFrame ni otros componentes de entrada/salida. Una clase main de demostración podrá utilizar consola, pero deberá permanecer separada del modelo.

E1-04 – Tripulación

La nave deberá tener una tripulación. Cada tripulante deberá poseer, como mínimo: identidad, cargo, planeta de origen y antigüedad.

Cargos admitidos:

- capitán;
- consejero;
- teniente;
- alférez.

Orígenes admitidos:

- terrícola;
- vulcano;
- marciano.

E1-05 – Bitácora

La Bitácora deberá registrar eventos relevantes, errores, cambios del Motor Warp, ejecución de misiones y operaciones sobre recursos. Los eventos deberán poder consultarse en orden temporal.

E1-06 – Tres misiones simplificadas mediante Template Method

Cada grupo implementará versiones simplificadas de M-01 (intercepción y asistencia), M-02 (recolección) y M-03 (retorno seguro). En esta entrega serán independientes del tablero, los sensores, los eventos dinámicos y la concurrencia.

Toda misión deberá seguir el siguiente ciclo general:

1. preparar;

2. ejecutar;
3. evaluar el resultado;
4. cerrar.

El ciclo general deberá implementarse mediante Template Method. Las variantes concretas podrán redefinir los pasos permitidos sin alterar el orden general del proceso.

Las tres misiones deberán actualizar recursos, registrar acontecimientos y producir un informe al finalizar. En E2, esas mismas clases deberán evolucionar incorporando navegación y universo dinámico; no se comenzará otra solución desde cero.

E1-07 – Creación de naves mediante Factory

El sistema deberá permitir crear naves exploradoras, cargueros y naves de combate mediante una fábrica.

El código cliente deberá solicitar el tipo requerido sin instanciar directamente las clases concretas. Todas las naves creadas deberán respetar una abstracción común y comenzar con una configuración válida de componentes y recursos.

E1-08 – Liquidación de haberes mediante Decorator

El sistema deberá calcular los haberes mensuales de cada tripulante mediante la composición de conceptos remunerativos.

Remuneración correspondiente al cargo:

- capitán: 1000 PG;
- consejero: 600 PG;
- teniente: 400 PG;
- alférez: 200 PG.

Adicional por cada año de antigüedad, calculado sobre la remuneración correspondiente al cargo:

- capitán: 20 %;
- consejero: 5 %;
- teniente: 3 %;
- alférez: 0,5 %.

El consejero recibirá además 2 PG por cada consejo registrado durante el período liquidado.

Subsidio mensual por origen:

- terrícola: 20 PG;
- vulcano: 30 PG;
- marciano: 18 PG.

El resultado deberá informar el total y mantener identificable el aporte de cada concepto. El cálculo deberá resolverse mediante Decorator y evitar una jerarquía formada por todas las combinaciones posibles de cargo, origen y antigüedad.

E1-09 – Recursos y mantenimiento

La nave deberá representar, como mínimo: combustible, energía, desgaste y necesidad o nivel de mantenimiento.

El sistema deberá permitir cargar combustible, cargar energía y realizar mantenimiento. Ninguna operación podrá dejar los recursos en un estado inválido.

E1-10 – Informe mínimo de misión

Al cerrar una misión deberá generarse un informe que incluya: misión ejecutada, resultado, acciones principales, recursos consumidos, estado final de la nave y observaciones relevantes.

4. Patrones obligatorios

Para cada uno de los patrones se exige que su implementación sea funcional, justificable y verificable en el dominio:

- State: Estados y transiciones del Motor Warp
- Factory: Creación de tipos de nave
- Template Method: Ciclo general de ejecución de una misión
- Decorator: Composición del haber por cargo, antigüedad, origen y conceptos particulares

No será suficiente incluir clases con el nombre del patrón; se evaluará el desacoplamiento conseguido y su correcta aplicación.

5. Observaciones

- La Nave no podrá tener combustible o energía negativos.
- El desgaste y el mantenimiento deberán permanecer dentro de los rangos definidos.
- El Motor Warp no aceptará transiciones inválidas silenciosas.
- Una Misión no podrá ejecutarse sin preparación previa.
- Una Misión no podrá cerrarse sin resultado e informe.
- La Bitácora no aceptará eventos nulos o vacíos.
- Las operaciones sobre recursos deberán conservar la consistencia de la nave.
- La antigüedad no podrá ser negativa y el cálculo de haberes no aceptará conceptos nulos ni importes inválidos.
- Toda nave creada por la fábrica deberá comenzar en un estado válido.
- Las clases del dominio no dependerán de consola ni de Swing.

Las precondiciones e invariantes se comprobarán cerca de las operaciones que protegen. Las excepciones se reservarán para errores que deban propagarse al coordinador o al código cliente; no sustituirán todas las decisiones normales del dominio.

6. Evidencias funcionales mínimas

El equipo deberá demostrar de forma reproducible:

- transiciones válidas e inválidas del Motor Warp;
- ejecución completa del ciclo de M-01, M-02 y M-03;
- creación de los tres tipos de nave mediante Factory;
- ausencia de instanciación directa de naves concretas desde el código cliente;
- modificación consistente de recursos y mantenimiento;

- generación del informe de misión;
- registro temporal de eventos en la Bitácora;
- cálculo verificable de haberes para los cuatro cargos y los tres orígenes, incluyendo al menos una composición de dos o más decoradores;
- al menos un caso de rechazo provocado por una precondition o invariante incumplida.

No se exige testing automatizado avanzado. Las evidencias podrán presentarse mediante pruebas automatizadas, ejecuciones controladas o demostraciones reproducibles, siempre que permitan comprobar inequívocamente el resultado.

7. Herramientas y entrega

- Lenguaje: Java.
- Construcción del proyecto: Maven obligatorio.
- Control de versiones: GitHub obligatorio.
- IDE: JDeveloper, IntelliJ IDEA o NetBeans.

El repositorio deberá contener: código fuente, estructura estándar de Maven, documentación del diseño, diagrama de clases, explicación breve de las responsabilidades y patrones, evidencias funcionales, y un README.md con instrucciones de compilación, ejecución y verificación.

Las consultas del modelo deberán devolver datos utilizables por distintos clientes, sin imprimirlos ni formatearlos directamente para consola o Swing. En Moodle se entregará la URL del repositorio GitHub.

8. Criterios mínimos de aprobación

- El proyecto compila y ejecuta según las instrucciones entregadas.
- Los diez requerimientos de E1 están implementados.
- Los cuatro patrones obligatorios se aplican al problema indicado.
- Los contratos centrales están explicitados y se utilizan para detectar estados inválidos.
- La Bitácora permite seguir la ejecución principal.
- La misión modifica recursos y produce su informe final.
- Las evidencias permiten verificar el comportamiento solicitado.
- El repositorio está organizado y contiene la documentación requerida.

III. Ficha de Inicio – Entrega 1

Versión: v0.1

1. Propósito

Esta ficha fija los datos comunes necesarios para comenzar la Entrega 1. Complementa el enunciado y evita que cada grupo invente valores diferentes.

2. Configuración inicial de las naves

Todos los recursos se expresan en unidades enteras.

Tipo de nave	Combustible inicial	Energía inicial	Desgaste inicial
Exploradora	60	80	0
Carguero	100	60	0
Combate	80	100	0

Para las tres naves:

- capacidad máxima de combustible: 100;
- capacidad máxima de energía: 100;
- desgaste permitido: de 0 a 100;
- requiere mantenimiento cuando el desgaste es 80 o superior;
- realizar mantenimiento lleva el desgaste a 0;
- estado inicial del Motor Warp: Disponible.

Una carga de combustible o energía no podrá superar la capacidad máxima. Una operación que no pueda completarse sin violar un límite deberá rechazarse sin modificar parcialmente la nave.

3. Tripulación mínima

Cada nave deberá comenzar con, al menos:

- un/a capitán/a;
- cuatro tripulantes adicionales de cualquiera de los cargos admitidos.

4. Misiones obligatorias

Cada grupo implementará las tres misiones siguientes:

Misión	Acción simplificada de E1	Condición de éxito
M-01 – Intercepción y asistencia	Llegar al objetivo simulado y realizar la asistencia	La asistencia se completa con recursos suficientes
M-02 – Recolección	Llegar al punto simulado y obtener datos o muestra	El elemento queda registrado como obtenido

Misión	Acción simplificada de E1	Condición de éxito
M-03 – Retorno seguro	Completar el regreso simulado	La nave finaliza en estado operativo válido

En E1 no se implementan el tablero, ruta, radares, snapshots ni eventos del universo. La misión representa una operación controlada y conserva el ciclo preparar → ejecutar → evaluar → cerrar.

5. Costos de la misión simplificada

Toda misión consume por su operación controlada:

- 4 unidades de combustible;
- 4 puntos de desgaste.

Al completar el objetivo se agrega la siguiente energía adicional:

Misión	Energía adicional
M-01 – Intercepción y asistencia	5
M-02 – Recolección	5
M-03 – Retorno seguro	0

Estos valores permiten ejercitar recursos e informes. En E2 serán reemplazados por el cálculo detallado según celdas, escaneos y acontecimientos del recorrido.

6. Escenarios mínimos que debe demostrar el grupo

Escenario A – Ejecución correcta

1. Crear mediante Factory una nave de cada tipo.
2. Seleccionar una y asignarle una tripulación válida.
3. Preparar, ejecutar, evaluar y cerrar M-01.
4. Repetir el ciclo completo con M-02 y M-03.
5. Mostrar para cada una los recursos finales, la Bitácora y el informe.

Escenario B – Recursos insuficientes

1. Utilizar una nave sin recursos suficientes para la misión.
2. Intentar prepararla o ejecutarla.
3. Verificar que la operación se rechaza, no deja cambios parciales y registra el motivo.

Escenario C – Motor Warp

1. Recorrer la secuencia válida Disponible → Preparando salto → En warp → Enfriamiento → Disponible.
2. Intentar al menos una transición inválida.

3. Verificar que la transición inválida se rechaza y queda registrada.

Escenario D – Contrato inválido

Intentar una carga o actualización que exceda una capacidad o deje un recurso fuera de rango. El estado anterior deberá conservarse.

7. Lista de comprobación antes de entregar

1. El proyecto compila y ejecuta mediante Maven.
2. Se crean los tres tipos de nave mediante Factory.
3. El Motor Warp aplica State y rechaza transiciones inválidas.
4. M-01, M-02 y M-03 aplican el mismo Template Method.
5. Las tres misiones completan sus cuatro etapas en el orden establecido.
6. Los recursos respetan capacidades e invariantes.
7. La Bitácora conserva los acontecimientos en orden temporal.
8. El cierre produce el informe requerido.
9. Los cuatro escenarios anteriores son reproducibles.
10. El repositorio incluye diagrama, explicación de patrones y README.md.
11. Todos los integrantes pueden explicar la solución.

Ante una diferencia, prevalecen el Enunciado de Entrega 1 y las comunicaciones formales de la cátedra.

IV. Tabla de Recursos, Costos y Consecuencias

Versión: v0.1

1. Criterio general

Todas las misiones utilizan las mismas reglas de consumo. El costo total depende principalmente de la cantidad de celdas recorridas y de los inconvenientes encontrados. No existe una tabla diferente para cada tipo de nave o misión.

Los escenarios comenzarán con valores de recursos indicados por la cátedra. Combustible y energía no podrán ser negativos; el desgaste se mantendrá entre 0 y 100.

2. Costos comunes

Antes de realizar una acción, el sistema debe comprobar que existan recursos suficientes. Impulso permite un avance por ciclo. Warp permite hasta dos, pero cada celda conserva el mismo costo de escaneo y movimiento. Si el segundo avance no puede ejecutarse, no se cobra su movimiento; sí quedan consumidos los recursos de toda observación o acción que efectivamente se haya realizado.

Acción	Combustible	Energía	Desgaste
Escaneo de largo alcance	0	5	0
Escaneo de corto alcance	0	1	0
Avanzar una celda	1	0	1
Recalcular una ruta	0	2	0

3. Consecuencias del universo

Próxima celda o condición	Respuesta obligatoria	Consecuencia adicional
Celda libre	Avanzar	Sólo el costo normal del movimiento
Asteroide	No ingresar y recalcular	Costo del nuevo cálculo y del recorrido alternativo
Anomalía gravitacional	Recalcular para evitar la región	Si no existe una ruta válida, abortar la misión
Tormenta electromagnética	Reducir velocidad	Consumir 2 unidades adicionales de energía mientras se atraviesa la región
Nave externa	Aplicar Double Dispatch	Registrar el protocolo; sin combate ni daño
Señal de auxilio	Registrar y consultar al capitán	Si autoriza el desvío, se aplican los costos normales de la nueva ruta
Destino de la misión	Ingresar y ejecutar el objetivo	Aplicar el costo específico indicado abajo

4. Acción final de cada misión

Misión	Acción al llegar	Costo adicional
M-01 Intercepción y asistencia	Verificar o asistir al objetivo	5 de energía
M-02 Recolección	Obtener los datos o la muestra	5 de energía
M-03 Retorno seguro	Confirmar el arribo	Sin costo adicional

5. Cálculo total

El costo de una misión se obtiene sumando:

1. escaneo de largo alcance;
2. escaneos de corto alcance realizados;
3. movimientos efectivamente ejecutados;
4. recálculos y desvíos;
5. consecuencias adicionales del entorno;
6. acción final de la misión.

No se asignará un costo fijo a toda la misión, porque el Motor de Simulación puede modificar el recorrido mientras se ejecuta.

6. Ejemplo breve

Una misión de recolección recorre cuatro celdas y no encuentra inconvenientes:

- largo alcance: 5 de energía;
- cuatro escaneos cortos: 4 de energía;
- cuatro movimientos: 4 de combustible y 4 de desgaste;
- recolección: 5 de energía.

Resultado: 4 de combustible, 14 de energía y 4 de desgaste.

Si aparece un asteroide, se agrega el costo de recalcular y los movimientos de la nueva ruta.

7. Regla de simplicidad

Los elementos del universo no producen cálculos complejos ni daños acumulativos. Cada elemento provoca una respuesta principal. La Bitácora debe registrar la observación, la decisión, los recursos consumidos y el resultado.

V. Reglas Operativas del TPG 2026

Versión: v0.3 — Fecha: 31/07/2026

1. Integración de los grupos

- El TPG se realizará en grupos de cuatro integrantes.
- El mismo grupo continuará durante Entrega 1 y Entrega 2.
- La conformación deberá registrarse por el medio indicado por la cátedra antes del inicio formal del trabajo.
- Los cambios de grupo requerirán autorización de la cátedra.
- Después de Entrega 1, los cambios sólo se admitirán de manera excepcional y deberán dejar constancia de cómo se reasignan las responsabilidades.
- La reducción o ampliación del grupo no modifica automáticamente el alcance del trabajo.

2. Responsabilidad del equipo

El grupo será responsable de:

- mantener un único proyecto incremental;
- integrar las contribuciones de sus integrantes;
- conservar el proyecto compilable;
- documentar las decisiones principales;
- verificar los requerimientos obligatorios;
- conocer y poder explicar la solución completa.

La división interna de tareas no limita la responsabilidad de cada integrante durante la defensa.

3. Repositorio GitHub

- El trabajo utilizará un único repositorio GitHub por grupo.
- Todos los integrantes deberán tener acceso al repositorio mediante sus cuentas personales.
- No se utilizará una cuenta compartida para realizar todas las contribuciones.
- El repositorio deberá contener un proyecto Maven reproducible.
- La rama principal deberá conservar una versión integrada y ejecutable.
- La rama de desarrollo (si existiere) y el trabajo en ramas y su posterior integración deberá dejar una historia comprensible.
- No se exige una cantidad mínima de commits. La cantidad de commits no se utilizará como medida automática de participación.
- Se evaluará la evidencia conjunta de commits, cambios, documentación, integración y defensa.

4. Identificación de las entregas

Cada entrega quedará identificada mediante una etiqueta Git:

- entrega-e1 para Entrega 1;

- entrega-e2 para Entrega 2.

La etiqueta deberá señalar el commit exacto que el grupo presenta para evaluación. En Moodle se entregará la URL del repositorio. La cátedra evaluará el contenido identificado por la etiqueta correspondiente. Los cambios posteriores al vencimiento no formarán parte de la entrega, salvo autorización expresa.

5. Contenido mínimo del repositorio

El repositorio deberá incluir obligatoriamente:

1. Código fuente.
2. pom.xml.
3. Pruebas o demostraciones reproducibles.
4. README.md.
5. Diagrama de clases.
6. Documentación de contratos y patrones.
7. Evidencias solicitadas por la matriz de casos.
8. docs/uso-ia.md.
9. database/schema.sql en Entrega 2.
10. .gitignore que excluya archivos generados, binarios y datos locales de H2.

El README.md deberá explicar detalladamente:

1. requisitos de ejecución;
2. comandos Maven necesarios;
3. forma de iniciar el sistema;
4. forma de ejecutar las verificaciones;
5. configuración local necesaria;
6. etiqueta correspondiente a la entrega.

6. Participación y autoría

Todos los integrantes deberán realizar contribuciones identificables. Estas podrán corresponder a:

- código;
- pruebas y casos de verificación;
- documentación técnica;
- integración;
- revisión y corrección de diseño.

Una gran cantidad de commits pequeños no demuestra por sí sola una contribución significativa. Del mismo modo, una contribución relevante no se descartará solamente por estar concentrada en pocos commits. La cátedra podrá utilizar el historial y la defensa para resolver dudas sobre participación o autoría.

7. Uso permitido de inteligencia artificial

Se permite utilizar herramientas de IA para:

- comprender conceptos;
- analizar alternativas de diseño;
- recibir explicaciones;
- sugerir pruebas o casos límite;
- asistir en la detección y corrección de errores;
- revisar documentación;
- proponer fragmentos de código que luego sean comprendidos, revisados e integrados por el equipo.

El equipo conserva la responsabilidad total sobre todo lo incorporado al repositorio.

8. Registro del uso de IA

El archivo docs/uso-ia.md deberá registrar los usos relevantes de IA indicando:

- herramienta utilizada;
- propósito de la consulta;
- componente o documento afectado;
- resultado aprovechado;
- revisión o modificación realizada por el equipo;
- forma en que se verificó el resultado.

No será necesario registrar cada autocompletado trivial. Deberán declararse los usos que hayan influido en el diseño, el código, las pruebas o la documentación entregada.

9. Usos no permitidos

Bajo ninguna circunstancia estará permitido:

1. presentar código o documentación que el equipo no pueda explicar;
2. utilizar IA para fabricar evidencias, ejecuciones o resultados inexistentes;
3. ocultar usos relevantes de IA;
4. utilizar herramientas de IA durante la defensa;
5. copiar soluciones de otros grupos;
6. incorporar código externo sin indicar su fuente y condiciones de uso.

10. Defensa

- La defensa será obligatoria.
- Participarán todos los integrantes.
- Tendrá una duración prevista de 20 minutos por grupo.
- Los primeros minutos se destinarán a una demostración breve y reproducible.
- El tiempo restante se utilizará para preguntas individuales y grupales.

- La cátedra podrá seleccionar casos de la matriz de verificación, solicitar explicaciones de diseño o pedir una modificación pequeña y acotada.
- Cada integrante deberá poder explicar el flujo general, los patrones obligatorios, los contratos y la parte del trabajo en la que participó.
- Durante la defensa no se permitirá utilizar IA.

11. Resultado de la defensa

La evaluación combinará: calidad del producto grupal, cumplimiento de requerimientos, evidencia de participación y comprensión individual demostrada.

Una defensa individual insuficiente no modificará automáticamente la evaluación de los demás integrantes. La cátedra podrá solicitar una nueva defensa individual, de acuerdo con las reglas generales de evaluación de la asignatura. La ausencia deberá justificarse por los mecanismos institucionales correspondientes. La cátedra definirá la instancia de defensa pendiente.

12. Integridad y fuentes externas

- Toda biblioteca adicional deberá declararse y justificarse.
- El código tomado o adaptado de una fuente externa deberá citarse.
- Deberán respetarse las licencias aplicables.
- No se admitirán repositorios derivados de otro grupo ni soluciones entregadas en años anteriores sin autorización.

13. Regla de prevalencia

Ante una contradicción, prevalecerán en este orden:

1. disposiciones institucionales y reglamento de la asignatura;
2. comunicaciones formales de la cátedra;
3. Documento Maestro y enunciado vigente;
4. estas reglas operativas;
5. README y documentación interna del equipo.

VI. Rúbrica – Entrega 1

1. Propósito

Esta rúbrica evalúa la Entrega 1 del TPG 2026. Cada criterio se vincula con los requerimientos E1-01 a E1-10 y debe corregirse mediante evidencias observables.

La rúbrica totaliza 100 puntos. Para cada criterio se aplicará al peso el factor correspondiente al nivel alcanzado: Excelente 100 %, Bien 80 %, Regular 60 %, Insuficiente 30 % y Ausente/no verificable 0 %.

2. Criterios y niveles de desempeño (en construcción)

Criterio	Peso	Excelente	Bien	Regular	Insuficiente
Modelo OO de Nave y Tripulación (E1-01, E1-04)	18	Responsabilidades claras, estado encapsulado, abstracción común de nave y relaciones de dominio coherentes. Los tres tipos de nave y la tripulación se integran sin duplicación relevante.	El modelo es correcto y comprensible, con acoplamientos o exposiciones menores.	El modelo funciona, pero mezcla responsabilidades, expone estado o presenta relaciones poco claras.	No existe un modelo OO reconocible o las clases son estructuras de datos sin comportamiento coherente.
Motor Warp mediante State (E1-02)	8	Los estados encapsulan el comportamiento y todas las transiciones válidas e inválidas son verificables y coherentes.	State está correctamente aplicado, con omisiones menores en transiciones o registros.	Existe una estructura semejante a State, pero parte del comportamiento permanece concentrado en condicionales externos.	No se aplica State o se reemplaza por un switch/if central que controla todos los estados.
Asistente de Comando y Bitácora (E1-03, E1-05)	10	El asistente coordina sin apropiarse de la lógica de los subsistemas; maneja errores y deja una bitácora temporal completa y significativa.	La coordinación y el registro son correctos, con omisiones menores.	Funciona parcialmente, pero el asistente concentra lógica o la bitácora registra información insuficiente.	El asistente no coordina correctamente o la bitácora es inexistente, genérica o no consultable.
Tres misiones mediante Template Method (E1-06)	12	M-01, M-02 y M-03 reutilizan el ciclo preparar-ejecutar-evaluar-cerrar; cada variante redefine sólo sus pasos particulares.	Las tres variantes y Template Method son correctos, con omisiones menores de extensibilidad o registro.	Las tres misiones existen, pero duplican parte del ciclo o alguna variante está incompleta.	Falta una misión, no existe un ciclo común o no se aplica Template Method.

Criterio	Peso	Excelente	Bien	Regular	Insuficiente
Creación de naves mediante Factory (E1-07)	7	La fábrica crea exploradora, carguero y combate mediante una abstracción común; el cliente no conoce clases concretas y un nuevo tipo puede agregarse con cambios localizados.	Los tres tipos se crean mediante Factory, con algún acoplamiento menor.	La fábrica existe, pero el cliente aún instancia o conoce clases concretas innecesariamente.	No se aplica Factory o sólo se encapsula un new sin desacoplar la creación.
Recursos, mantenimiento e informe (E1-08, E1-09)	15	Combustible, energía, desgaste y mantenimiento conservan invariantes; la misión actualiza el estado y genera un informe completo y consistente.	El manejo es correcto, con simplificaciones menores en recursos o informe.	El estado se actualiza parcialmente o el informe es incompleto.	Los recursos quedan inconsistentes o no se genera un informe útil.
Diseño por contrato y manejo de errores	14	Invariantes, precondiciones y postcondiciones son explícitas, coherentes y se utilizan para detectar y registrar fallas.	Los contratos cubren las piezas centrales, con algunas omisiones.	Existen contratos aislados o principalmente documentales, con poca aplicación efectiva.	No hay contratos relevantes o los estados inválidos se aceptan silenciosamente.
Verificación funcional	10	Todos los requerimientos cuentan con escenarios reproducibles, resultados esperados y evidencia suficiente, incluyendo casos válidos e inválidos.	Existe evidencia suficiente para verificar el flujo principal y la mayoría de los rechazos.	La evidencia es parcial, informal o difícil de reproducir.	No puede verificarse de manera clara el comportamiento entregado.
Calidad del repositorio y documentación	6	Proyecto Maven ordenado, historial Git coherente, README completo, diagrama consistente y decisiones de diseño justificadas.	El repositorio y la documentación permiten comprender, compilar y ejecutar el proyecto, con omisiones menores.	El proyecto puede ejecutarse, pero la organización o documentación dificulta su revisión.	No compila según lo informado, faltan entregables centrales o el repositorio no permite reconstruir el trabajo.

3. Condiciones mínimas de aprobación

Para aprobar E1 deberán cumplirse todas las siguientes condiciones:

- el proyecto compila y ejecuta según el README;

- los requerimientos E1-01 a E1-10 están implementados;
- State, Factory, Template Method y Decorator están presentes en la aplicación obligatoria indicada;
- las tres misiones completan su ciclo y generan informe;
- la Bitácora registra los eventos centrales;
- los recursos mantienen sus invariantes;
- existen contratos explícitos en las piezas centrales;
- existe evidencia funcional reproducible;
- el repositorio contiene código, documentación y diagrama de clases.

4. Límites de calificación por incumplimientos centrales

- Sin State real en el Motor Warp: Patrones/State no supera Insuficiente.
- Sin Factory aplicada a los tres tipos de nave: Factory no supera Regular.
- Con instanciación directa generalizada de naves concretas desde el cliente: Factory no supera Regular.
- Sin ciclo común reusable de misión: Template Method no supera Insuficiente.
- Si falta alguna de las tres misiones o su informe final: Misión y Recursos e informe no superan Regular.
- Sin contratos sobre Nave, Motor Warp, Misión y recursos: Diseño por contrato no supera Regular.
- Sin evidencias funcionales reproducibles: la entrega no puede alcanzar el nivel Bien global.
- Si el proyecto no compila o no puede ejecutarse según el README, la entrega no alcanza las condiciones mínimas de aprobación.

5. Evidencia mínima para la corrección

El equipo de estudiantes deberá tener preparados los siguientes elementos de demostración:

- Diagrama de clases consistente con el código.
- Escenario de creación de cada tipo de nave.
- Escenario de transiciones válidas e inválidas del Motor Warp.
- Ejecución completa de M-01, M-02 y M-03.
- Registro temporal en Bitácora.
- Estado de recursos antes y después de una misión y de una operación de mantenimiento.
- Informe mínimo generado al cierre.
- Caso de rechazo por incumplimiento de una precondition o invariante.
- Instrucciones reproducibles de compilación y ejecución.

6. Regla de corrección

La presencia nominal de una clase o interfaz no demuestra la aplicación de un patrón. La corrección deberá considerar el comportamiento, la estructura, el desacoplamiento conseguido y la evidencia de ejecución.